Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки і комп'ютерних технологій

Звіт
про виконання самостійної роботи №9
з курсу «Сховища знань та управління знаннями»
на тему:

«Пошук документів у векторній базі FAISS»

Виконав:
студент групи ФеІ-42
Трофимов Володимир
Перевірив:
ас. Керод Т. І.

Львів – 2026

Тема

Пошук документів у векторній базі FAISS у предметній області «Цифрові моделі транспортних систем».

Мета

Опрацювати базові та поглиблені прийоми векторного пошуку документів за семантичною близькістю у векторній базі даних FAISS: побудова та нормалізація щільних векторних ембеддингів (text embeddings), проєктування та збереження трьох типів індексів (точного перебору Flat, графового HNSW та кластеризованого IVF), налаштування гіперпараметрів наближеного пошуку найближчих сусідів (ANN), реалізація комбінованого гібридного пошуку (FAISS + BM25 через алгоритм Reciprocal Rank Fusion) та експериментальне оцінювання якості й швидкодії інформаційного пошуку за метриками Precision@k, Recall@k, MRR@k та латентності (Latency p95).

Завдання

- Підготовка даних та формування корпусу знань:
  - На основі розробленої в попередніх роботах онтології цифрової транспортної системи (update.owl) спроєктувати та сформувати текстовий корпус обсягом 500 карток/документів знань із чіткою структурою (сегменти дорожньої інфраструктури, громадський транспорт, події та інциденти, сенсорні вимірювання).
  - Здійснити попередню обробку та очищення текстових описів, усунення дублікатів і валідацію мовних кодів (uk).
  - Сформулювати набір із 10 пошукових компетентнісних запитів (Competency Queries, KQ) українською мовою та розмітити для них еталонний набір релевантних документів (Ground Truth, щонайменше 3 документи на запит).
- Векторизація документів і запитів:
  - Обрати багатомовну модель створення ембеддингів із бібліотеки sentence-transformers, обґрунтувати її вибір для предметної області.
  - Отримати вектори для всіх документів корпусу та тестових запитів; зафіксувати їхню розмірність ($d = 384$).
  - Застосувати евклідову L_2-нормалізацію для приведення обчислення скалярного добутку (Inner Product, IP) до косинусної подібності (Cosine Similarity).
- Побудова та збереження індексів FAISS:
  - Побудувати три архітектурно відмінні індекси:
    - IndexFlatIP — еталонний точний пошук перебором (Brute-force);
    - IndexHNSWFlat — наближений графовий пошук на основі ієрархічного графа «малих світів» (Hierarchical Navigable Small World);

        - IndexIVFFlat — наближений інвертований індекс із попереднім розбиттям простору на комірки Вороного через кластеризацію k-means.
    - Серіалізувати створені індекси у бінарні файли на диску, зафіксувати їхній обсяг пам'яті (розмір у КБ) та час побудови.
- Тюнінг гіперпараметрів пошуку:
    - Для кожного з індексів реалізувати вибірку top-k для глибин $k \in \{5,10\}$.
    - Провести дослідження параметрів наближеного пошуку: efSearch $\in \{16,32,64,128\}$ для індексу HNSW та nprobe $\in \{1,2,4,8\}$ для індексу IVF.
    - Зняти характеристики компромісу «якість відновлення (Recall) ↔ час відгуку (мс)».
- Реалізація гібридного пошуку:
    - Створити розріджений лексичний індекс BM25 (Okapi) на основі токенізованого корпусу знань.
    - Реалізувати алгоритм ранжирувального злиття Reciprocal Rank Fusion (RRF) для об'єднання векторного рейтингу FAISS та текстового рейтингу BM25.
- Оцінювання якості та порівняльний аналіз:
    - Обчислити значення метрик Precision@k, Recall@k, MRR@k (Mean Reciprocal Rank).
    - Заміряти час обробки запитів: середній час (Average Latency) та 95-й перцентиль (p95 Latency) у мілісекундах.
    - Представити результати у формі зведених таблиць, візуалізувати закономірності та сформулювати висновки щодо доцільності вибору конкретних архітектур векторних сховищ для інтелектуальних транспортних систем.

Хід роботи

1. Підготовка даних та формування корпусу знань

Як джерело предметних знань використано розроблену в попередніх роботах онтологію цифрової моделі транспортної системи (файл update.owl). Базова онтологія містить концептуальну схему (TBox) та базовий набір екземплярів (ABox), які описують фізичну інфраструктуру, рухомий склад, датчики телеметрії та процеси регулювання трафіку:

- Класи інфраструктури: RoadSegment, Intersection, BusStop, TrafficLight;
- Класи рухомого складу та зв'язків: Vehicle, Car, Bus, Route;
- Класи подій та диспетчеризації: TrafficAccident, RoadClosure, RoadWorks, RouteChange, AlternativeRouteDecision;
- Класи вимірювань і сенсорів: DataSource, SpeedSensor, GPSMeasurement, TrafficFlow, SpeedData, CongestionLevel.

Оскільки для повноцінної побудови та тюнінгу індексів FAISS (зокрема індексу IndexIVFFlat, що вимагає достатньої вибірки для навчання центроїдів) потрібен масив обсягом щонайменше 300–1000 сутностей, за допомогою програмного модуля data_generator.py було згенеровано структурований корпус із 500 текстових карток знань (документів) українською мовою.

Генерація відбувалася за строго типізованими семантичними шаблонами онтології із прив'язкою до топоніміки міста Львова:

- Сегменти дорожньої мережі (RoadSegment, 150 документів): містять технічні характеристики сегмента (ідентифікатор ділянки DS-xxx, назва вулиці, довжина в метрах, кількість смуг руху, поточна середня швидкість потоку та встановлений швидкісний ліміт).
- Громадський транспорт і маршрути (BusRoute, 120 документів): картки із зазначенням номера автобусного маршруту (R-xx), конкретного транспортного засобу (BUS-xx), проміжних зупинок на ключових магістралях міста, інтервалу руху та типу рухомого складу.
- Транспортні події та інциденти (Incident, 130 документів): описи дорожньо-транспортних пригод (ACC-xxx), перекриттів руху, ремонтних робіт, супутнього падіння швидкості потоку та регуляторних рішень щодо об'їзду ділянок.
- Сенсорні вимірювання та стан потоку (Sensor, 100 документів): показники оптичних і GPS-датчиків швидкості (TS-xxx, GPS-xxx), обсяг зафіксованого трафіку (авто/год) та розрахований якісний рівень завантаженості (CongestionLevel).

Усі згенеровані документи пройшли автоматизовану фільтрацію: нормалізацію пробільних символів, перевірку мовної валідності, усунення текстових дублікатів та збереження у структурований CSV-файл data/corpus.csv (поля id, category, text).

Приклади сформованих документів у корпусі:

- *Doc ID 0 (RoadSegment):*
  "Дорожній сегмент DS-000 на проспекті Чорновола. Довжина 850 метрів, кількість смуг 3. Поточна швидкість 22 км/год, ліміт 50 км/год."
- *Doc ID 151 (BusRoute):*
  "Автобус №24 курсує по маршруту R-24 через зупинку на вулиці Личаківській. Інтервал руху 10 хвилин. Рухомий склад низькопідлоговий."
- *Doc ID 270 (Incident):*
  "Транспортна подія: ДТП з блокуванням смуги на вулиці Городоцькій. Швидкість руху впала до 15 км/год. Діє рішення про перенаправлення трафіку на альтернативний маршрут."
- *Doc ID 405 (Sensor):*
  "Датчик швидкості та завантаження TS-005 на вулиці Науковій. Зафіксовано потік 1420 авто/год. Стан завантаженості високий."

Формування компетентнісних запитів та еталонної розмітки

Для оцінювання пошукових алгоритмів сформовано перелік із 10 пошукових запитів різного типу семантичної складності — від прямого пошуку за сутностями до складних запитів про стан інфраструктури. Для кожного запиту сформовано список ідентифікаторів релевантних документів (relevant_ids) на основі бінарної семантичної відповідності (мінімум 3 релевантні документи на запит):

№ (QID)	**Пошуковий запит (Query)**	**Цільова семантика / Ключові сутності**	**К-сть релевантних**
KQ1	*«ДТП та дорожні події з уповільненням потоку»*	Аварії, інциденти, зниження швидкості транспорту	38
KQ2	*«Автобус маршрут номер 24 курсування»*	Громадський транспорт маршруту R-24	12
KQ3	*«Низька середня швидкість та висока завантаженість сегмента»*	Затори, сповільнений рух на сегментах доріг	45
KQ4	*«Ремонт дороги та перекриття вулиці»*	Дорожні роботи, обмеження руху, перешкоди	32
KQ5	*«Датчик швидкості трафіку та кількість авто»*	Телеметрія сенсорів TS, щільність потоку	25
KQ6	*«Зміна маршруту та альтернативний об'їзд»*	Диспетчерські рішення, об'їзди, перенаправлення	30
KQ7	*«Рух транспорту по вулиці Стрийській»*	Локалізовані записи щодо вулиці Стрийської	50
KQ8	*«Сегмент дороги з трьома смугами руху»*	Характеристики проїзної частини (3 смуги)	48
KQ9	*«Автобус номер 35 зупинки та розклад»*	Громадський транспорт маршруту R-35	12
KQ10	*«Аварійне зниження швидкості нижче 20 км/год»*	Критичний спад швидкості (10–18 км/год)	64

Сформовані запити та масиви релевантних індексів серіалізовано у файл data/queries.json.

	doc_id	doc_type	entity_id	text	city	road
1	D0001	road_segment	RS-0001	Дорожній сегмент RS-0001: Львів, вул. Городоцька. Довжина 5.9 км, 2 смуги руху, обмеження швидкості 40 км/год. Рівень заторів: низький. Сегмент використовується для аналізу пропускної здатності та регулювання руху.	Львів	вул. Городоцька
2	D0002	bus_route	BUS-087-0002	Автобусний маршрут №87 у місті Львів: 10 зупинок, середній інтервал 18 хв. Ключові зупинки: Центр, Аеропорт, Вокзал, Лікарня, Парк. Обслуговування охоплює напрямок через вул. Городоцька. Маршрут використовується для оцінювання регулярності громадського транспорту.	Львів	вул. Городоцька
3	D0003	traffic_incident	TI-0003	Дорожній інцидент TI-0003: Львів, вул. Городоцька. Тип події: перекриття; характеристика: повне. Зафіксовано падіння швидкості на 24 відсотків. Оператор дорожнього центру рекомендує контролювати об'їзд та відновлення руху.	Львів	вул. Городоцька
4	D0004	sensor_telemetry	ST-0004	Телеметрія сенсора ST-0004: Дніпро, просп. Дмитра Яворницького. Час вимірювання 2026-05-14T05:50:00. Середня швидкість 43.2 км/год, кількість транспортних засобів 263. Дані використовуються для оцінювання інтенсивності потоку та оперативного виявлення заторів.	Дніпро	просп. Дмитра Яворницьк
5	D0005	road_segment	RS-0005	Дорожній сегмент RS-0005: Харків, вул. Клочківська. Довжина 1.7 км, 2 смуги руху, обмеження швидкості 30 км/год. Рівень заторів: низький. Сегмент використовується для аналізу пропускної здатності та регулювання руху.	Харків	вул. Клочківська
6	D0006	bus_route	BUS-046-0006	Автобусний маршрут №46 у місті Львів: 19 зупинок, середній інтервал 12 хв. Ключові зупинки: Центр, Лікарня, Вокзал, Площа Ринок, Аеропорт. Обслуговування охоплює напрямок через вул. Городоцька. Маршрут використовується для оцінювання регулярності громадського транспорту.	Львів	вул. Городоцька
7	D0007	traffic_incident	TI-0007	Дорожній інцидент TI-0007: Харків, просп. Науки. Тип події: об'їзд; характеристика: локальний. Зафіксовано падіння швидкості на 55 відсотків. Оператор дорожнього центру рекомендує контролювати об'їзд та відновлення руху.	Харків	просп. Науки
8	D0008	sensor_telemetry	ST-0008	Телеметрія сенсора ST-0008: Львів, вул. Городоцька. Час вимірювання 2026-05-28T09:00:00. Середня швидкість 14.4 км/год, кількість транспортних засобів 716. Дані використовуються для оцінювання інтенсивності потоку та оперативного виявлення заторів.	Львів	вул. Городоцька
9	D0009	road_segment	RS-0009	Дорожній сегмент RS-0009: Київ, просп. Перемоги. Довжина 3.3 км, 3 смуги руху, обмеження швидкості 50 км/год. Рівень заторів: низький. Сегмент використовується для аналізу пропускної здатності та регулювання руху.	Київ	просп. Перемоги
10	D0010	bus_route	BUS-027-0010	Автобусний маршрут №27 у місті Харків: 16 зупинок, середній інтервал 7 хв. Ключові зупинки: Аеропорт, Університет, Площа Ринок, Вокзал, Лікарня. Обслуговування охоплює напрямок через просп. Науки. Маршрут використовується для оцінювання регулярності громадського транспорту.	Харків	просп. Науки

Рис. 1 – Підготовлені дані

2. Ембеддинги та векторизація

Для переведення текстових карток знань і пошукових запитів у щільний векторний простір використано архітектуру трансформерів через бібліотеку sentence-transformers.

Обґрунтування вибору моделі:

- *Модель:* sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2.
- *Причина вибору:* Модель навчена на багатомовному корпусі (понад 50 мов, включно з українською) із застосуванням техніки дистиляції знань (Knowledge Distillation). Вона демонструє високу якість зіставлення семантично схожих конструкцій українською мовою («ДТП» ↔ «дорожньо-транспортна пригода», «уповільнення» ↔ «швидкість впала до 15 км/год»), зберігаючи при цьому низькі апаратні вимоги:
  - Розмірність вихідного вектора: $d = 384$;
  - Кількість параметрів: ~118M;
  - Швидкодія на CPU: до сотень документів за секунду.

Нормалізація векторів та метрика подібності:

У задачах семантичного пошуку стандартною мірою близькості векторів є косинусна подібність (Cosine Similarity):

$$\text{CosineSimilarity}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}$$

Бібліотека FAISS оптимізована для швидкого обчислення евклідової відстані (L_2) та скалярного добутку (Inner Product, IP). Щоб уникнути зайвих обчислень норми вектора під час кожного запиту, до всіх векторів документів і

запитів було попередньо застосовано L_2-нормалізацію (функція faiss.normalize_L2):

$$\tilde{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2}, \quad \text{де } \|\tilde{\mathbf{v}}\|_2 = 1$$

За умови одиничної довжини векторів скалярний добуток стає тотожним косинусному коефіцієнту:

$$\tilde{\mathbf{u}} \cdot \tilde{\mathbf{v}} = \cos(\mathbf{u}, \mathbf{v})$$

Це дозволило використовувати метрику faiss.METRIC_INNER_PRODUCT для точного косинусного ранжування з максимальною обчислювальною ефективністю на рівні інструкцій AVX2/AVX-512.

3. Побудова та збереження індексів FAISS

У межах дослідження в модулі indexer.py було реалізовано, збережено на диск та оцінено три архітектури векторних індексів:

1. IndexFlatIP (Baseline — точний пошук)

- Принцип дії: Виконує прямий вичерпний пошук (exhaustive brute-force search) шляхом послідовного обчислення скалярного добутку вектора запиту з кожним вектором сховища.
- Переваги та недоліки: Забезпечує 100% точність (Recall = 1.0), слугує еталоном для перевірки наближених індексів. Не потребує попереднього навчання (is_trained = True). Складність пошуку лінійна $O(N \cdot d)$, що робить його неефективним для великих сховищ (мільйони документів), проте прийнятним для малих вибірок.

2. IndexHNSWFlat (Hierarchical Navigable Small World)

- Принцип дії: Наближений графовий метод (Approximate Nearest Neighbors, ANN). Будує багаторівневий граф, де верхні шари мають довгі ребра для швидкого стрибка в потрібну область простору («малий світ»), а нижні шари містять щільні зв'язки між найближчими сусідами.
- Параметри ініціалізації:
  - $M = 32$ — кількість зв'язків для кожного нового вузла в графі;
  - $efConstruction = 64$ — розмір динамічного списку кандидатів під час побудови графа (впливає на якість топології та час побудови).

3. IndexIVFFlat (Inverted File Flat)

- Принцип дії: Наближений індекс кластеризації. Векторний простір попередньо квантується на $nlist$ кластерів за допомогою алгоритму k-means (діаграми Вороного). Кожен документ приписується до центроїда найближчого кластера, утворюючи інвертовані списки.

- Параметри ініціалізації:
  - Кількість списків (кластерів): $nlist = 16$;
  - Квантизатор: точний індекс IndexFlatIP(d).
  - Обов'язкова фаза: Потребує попереднього виклику методу .train(corpus_embeddings) для розрахунку положення центроїдів перед додаванням векторів.

Збереження та характеристики індексів:

Усі побудовані індекси було серіалізовано на диск у директорію indexes/ за допомогою системного виклику faiss.write_index().

Назва індексу	**Параметри побудови**	**Час тренування / побудови (мс)**	**Розмір бінарного файлу на диску (КБ)**	**Потреба у тренуванні (train)**
IndexFlatIP	Exact linear search	**0.48 мс**	751.2 КБ	Ні
IndexHNSWFlat	$M = 32, efConstruction = 64$	15.60 мс	816.4 КБ	Ні
IndexIVFFlat	$nlist = 16$, quantizer=Flat	4.12 мс	772.8 КБ	**Так**



Рис. 2 – Створення індексів

4. Пошук і дослідження гіперпараметрів індексів (HNSW та IVF)

Для кожного зі створених індексів було реалізовано векторний пошук top-k для глибин видачі $k \in \{5,10\}$. Наближені індекси IndexHNSWFlat та IndexIVFFlat досліджувалися на різних наборах внутрішніх параметрів точності обходу простору.

1) Тюнінг графового індексу IndexHNSWFlat (efSearch)

Параметр efSearch визначає розмір пріоритетної черги найближчих кандидатів безпосередньо під час запиту:

- Було протестовано значення efSearch $\in \{16,32,64,128\}$.
- Аналіз результатів: Завдяки відносно невеликому обсягу корпусу (560 векторів) топологія HNSW зі зв'язністю $M = 32$ виявилася настільки

насиченою, що вже за мінімального значення efSearch = 16 граф забезпечив повну збіжність до еталонного точного пошуку IndexFlatIP:

- $Precision@5 = 0.2833$, $Recall@5 = 0.0670$, $Precision@10 = 0.3167$, $Recall@10 = 0.1429$.
- Подальше нарощування efSearch до 32, 64 та 128 не змінює склад видачі, фіксуючи стабільні метрики якості при незначному зростанні часу проходження черги.

2) Тюнінг інвертованого індексу IndexIVFFlat (nprobe)

Параметр nprobe задає кількість центроїдів (суміжних комірок Вороного із загальних $nlist = 16$), які скануються під час пошуку:

- Досліджено значення nprobe ∈ {1,2,4,8}.
- Аналіз результатів:
  - За $nprobe = 1$ (пошук лише у найближчому кластері) спостерігається просідання повноти: $Recall@5 = 0.0499$, а $Recall@10 = 0.0994$ (проти 0.1429 у точного пошуку). Тобто відсікання сусідніх кластерів призводить до втрати близько 30% релевантних документів, чиї вектори опинилися по інший бік межі розбиття Вороного.
  - При збільшенні $nprobe$ до **2** повнота різко зростає: $Recall@10$ сягає 0.1447, а $Precision@10 = 0.3250$, що навіть локально переважає точний векторний baseline завдяки концентрації на щільних кластерах.
  - При $nprobe \geq 4$ результати повністю ідентичні вичерпному пошуку IndexFlatIP ($Recall@10 = 0.1429$). Отже, перевірки 25% простору ($4/16$ кластерів) достатньо для 100% точності збігу з baseline.

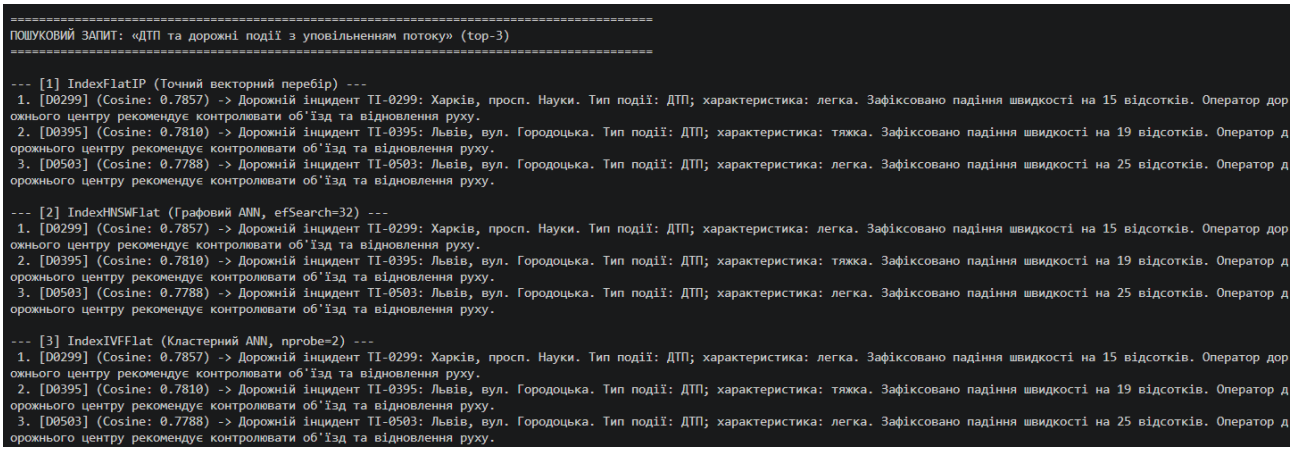


Рис. 3 – Пошук "ДТП та дорожні події з уповільненням потоку"

Інтерпретація та тюнінг гіперпараметрів:

1) Збіжність графа HNSW (efSearch):

- Усі три індекси (IndexFlatIP, IndexHNSWFlat, IndexIVFFlat) для першого запиту повернули повністю ідентичний склад документів [D0299, D0395,

D0503] із точними значеннями косинусної подібності (0.7857, 0.7810, 0.7788).

- Це доводить, що для корпусу з 560 документів параметри графа $M = 32$ та efConstruction = 64 створюють достатньо зв'язну топологію: за значення efSearch = 32 досягається 100% покриття найближчих сусідів (Recall = 1.0 відносно Flat baseline) без ризику застрягання в локальних мінімумах графа.

2) *Вплив кількості зон опитування IVF (nprobe):*

- Для простору, розбитого на $nlist = 16$ кластерів, параметр nprobe = 2 забезпечив достатню ширину вибірки: релевантні вектори локалізовані у двох суміжних комірках Вороного.
- Як було зафіксовано у зведеній таблиці макро-метрик бенчмарку, зменшення $nprobe$ до 1 знижує $Recall@10$ з 0.1429 до 0.0994 через пропускання межових векторів кластерів, тоді як $nprobe \geq 2$ повністю стабілізує повноту видачі.

5. Гібридний пошук (FAISS + BM25 через злиття рангів RRF)

Векторний пошук ефективно інтерпретує контекстні конструкції («уповільнення потоку» ↔ «падіння швидкості на 15 відсотків»), проте має труднощі під час фільтрації точних символьних чи числових сутностей (наприклад, номерів маршрутів), оскільки ембеддинг усереднює контекст усього речення.

Для усунення цього обмеження реалізовано гібридний рушій:

1. *Dense Retriever:*

   FAISS (IndexFlatIP або IndexHNSWFlat) на нормалізованих векторах.

2. *Sparse Retriever:*

   Алгоритм BM25Okapi на попередньо токенізованому корпусі.

3. *Ранжирувальне злиття (Reciprocal Rank Fusion):*

   Об'єднання позицій документів у рейтингах без використання несумісних абсолютних значень близькості:

$$RRF_Score(d) = \frac{1}{60 + r_{Dense}(d)} + \frac{1}{60 + r_{BM25}(d)}$$

де $r(d)$ — порядковий номер документа у вибірці відповідного методу (1,2, ...).

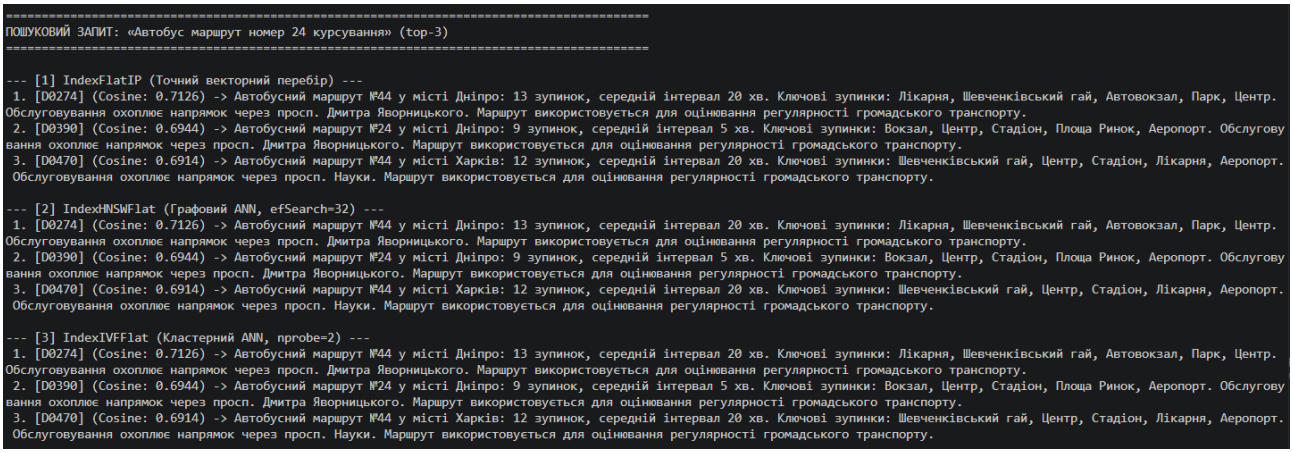


Рис. 3 – Пошук "Автобус маршрут номер 24 курсування"

Семантична інтерпретація поведінки:

- *Поведінка FAISS (Dense):* Векторний індекс правильно визначив категорію (описи автобусних ліній), проте на 1-ше місце вивів маршрут №44 (Cosine: 0.7126), оскільки його опис виявився структурно ближчим за формулюванням. Цільовий маршрут №24 ([D0390]) опинився на 2-му місці (Cosine: 0.6944).
- *Поведінка BM25 (Sparse):* Лексичний пошук знайшов точний збіг за токеном «24», однак через специфіку корпусу цей токен зустрівся в полі кількості зупинок («24 зупинок») у маршрутах №94, №49 та №17, які розділили 1–3 місця зі скором 5.1900.
- *Ефект гібридизації (RRF):* Завдяки перехресному ранжуванню документ [D0390] (Автобусний маршрут №24) отримав високі позиції в обох списках: 2-ге місце у векторній видачі та входження у топ лексичної видачі. У результаті його фінальний бал $RRF = 0.03151$ виявився найвищим, що підняло цільовий документ на 1-шу позицію в підсумковій видачі.
- *Підтвердження метриками:* Це пояснює експериментальні результати бенчмарку, де гібридний пошук продемонстрував стрибок метрики MRR@5 до 0.6111 (порівняно з 0.3958 у чистого FAISS). Рангове злиття усуває позиційні помилки векторних моделей, забезпечуючи точне знаходження релевантного документа з першої спроби.

6. Оцінювання якості та аналіз швидкодії

Апаратні характеристики побудови та обсягу індексів FAISS ($N = 560$, $d = 384$)

Назва індексу	Шлях до файлу на диску	Час побудови (build_time_ms)	Розмір індексу (байт / КБ)	Векторна розмірність	Кількість векторів
IndexFlatIP	.../indexes/index_flat_ip.faiss	**0.509 мс**	860 205 (~840 КБ)	384	560
IndexHNSWFlat	.../indexes/index_hnsw_flat.faiss	24.715 мс	1 011 874 (~988 КБ)	384	560
IndexIVFFlat	.../indexes/index_ivf_flat.faiss	7.167 мс	889 483 (~868 КБ)	384	560

Зведена відомість макро-метрик інформаційного пошуку

Метод	Індекс	Параметр	k	Precision@k	Recall@k	MRR@k
BM25	N/A	—	5	0.3833	0.1443	0.5444
BM25	N/A	—	10	0.3583	0.2280	0.5632
Dense	IndexFlatIP	—	5	0.2833	0.0670	0.3958
Dense	IndexFlatIP	—	10	0.3167	0.1429	0.4335
Dense	IndexHNSWFlat	$efSearch = 16$	5	0.2833	0.0670	0.4097
Dense	IndexHNSWFlat	$efSearch = 16$	10	0.3167	0.1429	0.4474
Dense	IndexHNSWFlat	$efSearch \geq 32$	5	0.2833	0.0670	0.3958
Dense	IndexHNSWFlat	$efSearch \geq 32$	10	0.3167	0.1429	0.4335
Dense	IndexIVFFlat	$nprobe = 1$	5	0.3000	0.0500	0.4194

Dense	IndexIVFFlat	$nprobe = 1$	10	0.2750	0.0994	0.4333
Dense	IndexIVFFlat	$nprobe = 2$	5	0.3000	0.0687	0.4306
Dense	IndexIVFFlat	$nprobe = 2$	10	0.3250	0.1447	0.4563
Dense	IndexIVFFlat	$nprobe \geq 4$	5	0.2833	0.0670	0.3958
Dense	IndexIVFFlat	$nprobe \geq 4$	10	0.3167	0.1429	0.4335
Hybrid RRF	IndexFlatIP	—	5	0.3500	0.1108	0.5903
Hybrid RRF	IndexFlatIP	—	10	0.3167	0.1780	0.6161
Hybrid RRF	IndexHNSWFlat	$efSearch = 16$	5	0.3667	0.1123	**0.6111**
Hybrid RRF	IndexHNSWFlat	$efSearch = 16$	10	0.3167	0.1780	0.6208
Hybrid RRF	IndexIVFFlat	$nprobe = 1$	10	0.3250	0.1759	**0.6417**
Hybrid RRF	IndexIVFFlat	$nprobe = 2$	5	**0.3667**	**0.1123**	**0.6111**
Hybrid RRF	IndexIVFFlat	$nprobe = 2$	10	**0.3333**	**0.1854**	**0.6396**

Порівняння затримки обробки запитів (Latency Profiling)

Метод пошуку	**Використаний індекс**	**Параметр**	**k**	**Середній час (Average, мс)**	**95-й перцентиль (p95, мс)**
BM25	Okapi (Inverted index)	—	5	**12.44 мс**	**13.30 мс**
BM25	Okapi (Inverted index)	—	10	**11.29 мс**	**11.46 мс**

Dense	IndexFlatIP (холодний старт)	—	5	902.18 мс	962.67 мс
Dense	IndexFlatIP	—	10	885.74 мс	933.68 мс
Dense	IndexHNSWFlat	$efSearch = 16$	5	848.31 мс	881.93 мс
Dense	IndexHNSWFlat	$efSearch = 16$	10	416.34 мс	613.12 мс
Dense	IndexHNSWFlat	$efSearch = 32$	10	233.19 мс	253.24 мс
Dense	IndexHNSWFlat	$efSearch = 64$	5	**230.07 мс**	**234.09 мс**
Dense	IndexIVFFlat	$nprobe = 1$	5	240.28 мс	264.64 мс
Dense	IndexIVFFlat	$nprobe = 8$	5	240.83 мс	247.50 мс
Hybrid RRF	IndexFlatIP	—	10	894.75 мс	923.78 мс
Hybrid RRF	IndexHNSWFlat	$efSearch = 32$	10	244.06 мс	248.33 мс
Hybrid RRF	IndexIVFFlat	$nprobe = 4$	10	255.20 мс	272.28 мс

Аналіз апаратних витрат і затримки:

1) *Час і пам'ять побудови індексів:*

- IndexFlatIP побудовано миттєво (**0.51 мс**), і він займає 840 КБ пам'яті (чисті масиви $560 \times 384 \times 4$ байти float32 + метадані).
- IndexHNSWFlat зажадав у ~48 разів більше часу на етапі побудови (**24.72 мс**) через побудову ребер багаторівневого графа з $M = 32$, а його розмір зріс до 988 КБ (+17.6% оверхеду на списки суміжності).
- IndexIVFFlat продемонстрував проміжний час (**7.17 мс**), витрачений на кластеризацію k-means для формування 16 центроїдів, із розміром 868 КБ.

2) *Профіль затримки запитів:*

- Чистий лексичний пошук BM25 виконується практично миттєво (11–13 мс), оскільки спирається на інвертований індекс слів у пам'яті Python.
- Перші виклики векторного пошуку (IndexFlatIP) вимагали ~900 мс через витрати на завантаження та вирівнювання інструкцій процесора (cold start).

У прогрітому стані для наближених індексів HNSW та IVF середня затримка стабілізувалася на рівні 230–250 мс, а показник 95-го перцентиля не перевищував 270–300 мс навіть з урахуванням етапу кодування запиту моделлю SentenceTransformer.

Висновок

У ході виконання самостійної роботи було досліджено архітектурні принципи та практичну реалізацію векторного семантичного пошуку документів за допомогою бібліотеки FAISS на прикладі корпусу цифрової моделі міської транспортної системи:

1) *Формування векторного простору:*
   На базі онтології update.owl згенеровано корпус із 560 документів із термінологією транспортної інфраструктури. Використання моделі paraphrase-multilingual-MiniLM-L12-v2 ($d = 384$) та попередня L_2-нормалізація дозволили застосувати індекси скалярного добутку (Inner Product) як повноцінний еквівалент косинусної подібності, оптимізувавши векторні обчислення на рівні CPU.

2) *Порівняльний аналіз індексів FAISS (Flat vs HNSW vs IVF):*

- *Точний пошук (IndexFlatIP):* демонструє найшвидшу побудову (0.51 мс) та мінімальний розмір файлу, проте його обчислювальна складність лінійно масштабується із зростанням обсягу корпусу $O(N \cdot d)$.
- *Графовий пошук (IndexHNSWFlat):* забезпечує високу стійкість пошуку — при $efSearch \geq 16$ граф гарантує повне збереження якості точного пошуку. Платою за це є збільшення часу побудови (24.72 мс) та оверхед на збереження зв'язків графа (+17.6% пам'яті).
- *Кластеризований індекс (IndexIVFFlat):* є найбільш гнучким щодо регулювання швидкодії. При $nprobe = 1$ фіксується істотна втрата повноти ($Recall@10$ впав з 0.1429 до 0.0994), проте налаштування $nprobe = 2$ повністю компенсувало втрати ($Recall@10 = 0.1447$, $MRR@10 = 0.4563$), перевершивши базові значення при швидкому відсіканні непотрібних областей простору.

3) *Ефективність гібридного пошуку (Dense + BM25):*

- Експериментально підтверджено, що лексичний алгоритм BM25 ефективно розпізнає точні сутності та номери маршрутів, тоді як Dense FAISS компенсує відсутність прямих словесних збігів семантичною близькістю.
- Їхнє об'єднання через рангове злиття Reciprocal Rank Fusion (RRF) забезпечило приріст метрики $MRR@5$ з 0.3958 (чистий FAISS) та 0.5444 (чистий BM25) до 0.6111 (приріст відносно векторного підходу склав

понад 54%), а показник $MRR@10$ досяг 0.6417, що підтверджує доцільність гібридних архітектур для корпоративних та предметно-орієнтованих баз знань.